



Git হলো একটি ওপেন সোর্স ডিস্ট্রিবিউটেড ভার্সন কন্ট্রোল সিস্টেম, যা আপনার কম্পিউটারে স্থানীয়ভাবে GitHub সম্পর্কিত সকল কাজের জন্য দায়ী। এই চিট শিটে সবচেয়ে গুরুত্বপূর্ণ এবং সাধারণভাবে ব্যবহৃত Git কমান্ডগুলো সহজ রেফারেন্সের জন্য অন্তর্ভুক্ত করা হয়েছে।

▶ ইনস্টলেশন ও GUI সমূহ

Git-এর জন্য প্ল্যাটফর্মভিত্তিক ইনস্টলারসহ, GitHub আপনাকে সর্বশেষ কমান্ড লাইন টুল ভার্সনে আপ-টু-ডেট থাকার সুবিধা প্রদান করে এবং দৈনন্দিন ব্যবহারের জন্য একটি গ্রাফিক্যাল ইউজার ইন্টারফেস (GUI) সরবরাহ করে, যা ইন্টারঅ্যাকশন, রিভিউ এবং রিপোজিটরি সিঙ্ক্রোনাইজেশনে সহায়তা করে।

Windows-এর জন্য GitHub:

- <https://windows.github.com>

Mac-এর জন্য GitHub:

- <https://mac.github.com>

Linux এবং Solaris প্ল্যাটফর্মের জন্য, সর্বশেষ রিলিজটি Git-এর অফিসিয়াল ওয়েবসাইটে পাওয়া যায়:

সকল প্ল্যাটফর্মের জন্য Git:

- <http://git-scm.com>

▶ সেটআপ (SETUP)

সকল লোকাল রিপোজিটরিতে ব্যবহৃত ইউজার ইনফরমেশন কনফিগার করা

→ এমন একটি নাম সেট করুন যা ভার্সন হিস্টোরি রিভিউয়ের সময় ক্রেডিট দেওয়ার জন্য সহজে শনাক্তযোগ্য হবে।

- `git config --global user.name "[firstname lastname]"`

→ এমন একটি ইমেইল ঠিকানা সেট করুন যা প্রতিটি হিস্টোরি মার্কারের সঙ্গে সংযুক্ত থাকবে।

- `git config --global user.email "[valid-email]"`

→ Git কমান্ড লাইনকে স্বয়ংক্রিয়ভাবে রঙিন করে তুলুন যাতে রিভিউ করা আরও সহজ হয়।

- `git config --global color.ui auto`

▶ সেটআপ ও ইনিশিয়ালাইজ (SETUP & INIT)

ইউজার তথ্য কনফিগার করা, রিপোজিটরি ইনিশিয়ালাইজ ও ক্লোন করা

→ একটি বিদ্যমান ডিরেক্টরিকে Git রিপোজিটরি হিসেবে ইনিশিয়ালাইজ করুন।

- `git init`

→ কোনো হোস্ট করা লোকেশন (যেমন GitHub)-এর URL ব্যবহার করে সম্পূর্ণ একটি রিপোজিটরি ডাউনলোড করে আনুন।

- `git clone [url]`

▶ স্টেজ ও স্ন্যাপশট (STAGE & SNAPSHOT)

→ আপনার ওয়ার্কিং ডিরেক্টরিতে কোন ফাইলগুলো পরিবর্তিত হয়েছে এবং কোনগুলো স্টেজ করা আছে তা দেখায় (পরবর্তী কমিটের জন্য প্রস্তুত)।

- `git status`

→ ফাইলটিকে এখন যেমন আছে, তেমনভাবে আপনার পরবর্তী কমিটের জন্য স্টেজ করুন।

- `git add [file]`



→ ফাইলটিকে আনস্টেজ করুন, কিন্তু ওয়ার্কিং ডিরেক্টরির পরিবর্তনগুলো রেখে দিন।

- `git reset [file]`

→ যেসব পরিবর্তন করা হয়েছে কিন্তু এখনো স্টেজ করা হয়নি, সেগুলোর পার্থক্য দেখায়।

- `git diff`

→ যেসব পরিবর্তন স্টেজ করা হয়েছে কিন্তু এখনো কমিট করা হয়নি, সেগুলোর পার্থক্য দেখায়।

- `git diff --staged`

→ স্টেজ করা কনটেন্টকে একটি নতুন কমিট স্ম্যাপশট হিসেবে সংরক্ষণ করুন।

- `git commit -m "[descriptive message]"`

▶ **ব্রাঞ্চ ও মার্জ (BRANCH & MERGE)**

ব্রাঞ্চে কাজ আলাদা করা, কনটেন্ট পরিবর্তন করা এবং পরিবর্তনগুলো একত্রিত করা

→ আপনার ব্রাঞ্চগুলো তালিকা আকারে দেখায়। বর্তমানে সক্রিয় ব্রাঞ্চের পাশে একটি * চিহ্ন প্রদর্শিত হবে।

- `git branch`

→ বর্তমান কমিটে একটি নতুন ব্রাঞ্চ তৈরি করুন।

- `git branch [branch-name]`

→ অন্য একটি ব্রাঞ্চে সুইচ করুন এবং সেটিকে আপনার ওয়ার্কিং ডিরেক্টরিতে চেকআউট করুন।

- `git checkout`

→ নির্দিষ্ট করা ব্রাঞ্চের ইতিহাসটি বর্তমান ব্রাঞ্চে মার্জ করুন।

- `git merge [branch]`

→ বর্তমান ব্রাঞ্চের ইতিহাসে সব কমিট দেখায়।

- `git log`

▶ **পরীক্ষা ও তুলনা (INSPECT & COMPARE)**

লগ, ডিফ ও অবজেক্ট তথ্য পর্যালোচনা করা

→ বর্তমানে সক্রিয় ব্রাঞ্চের কমিট ইতিহাস দেখায়।

- `git log`

→ branchA-তে যেসব কমিট আছে কিন্তু branchB-তে নেই, সেগুলো দেখায়।

- `git log branchB..branchA`

→ নির্দিষ্ট ফাইলটির ইতিহাস দেখায়, এমনকি ফাইলের নাম পরিবর্তন হলেও সেই ইতিহাস অনুসরণ করে।

- `git log --follow [file]`

→ branchA-তে যেসব পরিবর্তন আছে কিন্তু branchB-তে নেই, সেগুলোর পার্থক্য (diff) দেখায়।

- `git diff branchB...branchA`

→ নির্দিষ্ট SHA (কমিট/অবজেক্ট আইডি) দ্বারা যেকোনো Git অবজেক্ট মানব-পাঠযোগ্য (human-readable) ফরম্যাটে দেখায়।

- `git show [SHA]`

▶ **পথ পরিবর্তনের ট্র্যাক রাখা (TRACKING PATH CHANGES)**

ফাইল মুছে ফেলা এবং পথ পরিবর্তনের ভার্শন কন্ট্রোল করা

→ প্রজেক্ট থেকে ফাইলটি মুছে ফেলুন এবং এই অপসারণটিকে কমিট করার জন্য স্টেজ করুন।

- `git rm [file]`

GIT CHEAT SHEET

→ বিদ্যমান ফাইলের পথ পরিবর্তন করুন এবং এই মুভ অপারেশনটি সেটজ করুন।

- `git mv [existing-path] [new-path]`

→ সব কমিট লগ দেখায় এবং কোন কোন ফাইলের পথ (path) পরিবর্তিত হয়েছে তার ইঙ্গিতও দেয়।

- `git log --stat -M`

▶ পথ পরিবর্তনের ট্র্যাক রাখা (TRACKING PATH CHANGES)

অজান্তে কোনো ফাইল সেটজ বা কমিট হওয়া প্রতিরোধ করা

→ সিস্টেম-ওয়াইড ইগনোর প্যাটার্ন নির্ধারণ করে, যা সব লোকাল রিপোজিটরিতে প্রযোজ্য হবে।

- `git config --global core.excludesfile [file]`

উদাহরণস্বরূপ .gitignore ফাইলে এই প্যাটার্নগুলো রাখতে পারেন:

```
logs/
*.notes
pattern*/
```

এই ধরনের প্যাটার্ন সহ একটি .gitignore ফাইল তৈরি করুন যাতে নির্দিষ্ট ফাইল বা ফোল্ডার Git দ্বারা ট্র্যাক না হয়।

▶ পথ পরিবর্তনের ট্র্যাক রাখা (TRACKING PATH CHANGES)

অন্য রিপোজিটরি থেকে আপডেট গ্রহণ এবং লোকাল রিপোজিটরি আপডেট করা

→ একটি Git URL-কে একটি উপনাম (alias) দিয়ে যুক্ত করুন।

- `git remote add [alias] [url]`

→ নির্দিষ্ট রিমোট রিপোজিটরি থেকে সব ব্রাঞ্চ ডাউনলোড করুন (লোকাল ব্রাঞ্চে মার্জ না করে)।

- `git fetch [alias]`

→ রিমোটের নির্দিষ্ট ব্রাঞ্চে আপনার বর্তমান ব্রাঞ্চে মার্জ করে আপডেট করুন।

- `git merge [alias]/[branch]`

→ লোকাল ব্রাঞ্চের কমিটগুলো নির্দিষ্ট রিমোট রিপোজিটরির ব্রাঞ্চে পাঠান।

- `git push [alias] [branch]`

→ ট্র্যাক করা রিমোট ব্রাঞ্চ থেকে নতুন কমিটগুলো ফেচ ও মার্জ করে নিন (এক ধাপে আপডেট)।

- `git pull`

▶ ইতিহাস পুনর্লিখন (REWRITE HISTORY)

ব্রাঞ্চ পুনর্লিখন, কমিট আপডেট এবং ইতিহাস পরিষ্কার করা

→ বর্তমান ব্রাঞ্চের যেকোনো নতুন কমিট নির্দিষ্ট ব্রাঞ্চের পরে প্রয়োগ করুন (কমিট ইতিহাস সাজিয়ে নেয়)।

- `git rebase [branch]`

→ স্টেজিং এরিয়া ও ওয়ার্কিং ট্রি সম্পূর্ণভাবে মুছে ফেলে নির্দিষ্ট কমিট থেকে পুনর্লিখন করুন (সতর্কভাবে ব্যবহার করুন — পরিবর্তন হারিয়ে যেতে পারে)।

- `git reset --hard [commit]`



► ইতিহাস পুনর্লিখন (REWRITE HISTORY)

ব্রাঞ্চ পুনর্লিখন, কমিট আপডেট এবং ইতিহাস পরিষ্কার করা

→ পরিবর্তিত ও সেভ করা ফাইলগুলো অস্থায়ীভাবে সংরক্ষণ করুন।

- `git stash`

→ স্ট্যাশ করা পরিবর্তনগুলোর স্ট্যাক অর্ডারে তালিকা দেখান।

- `git stash list`

→ স্ট্যাশ স্ট্যাকের শীর্ষে থাকা পরিবর্তনগুলো পুনরায় ওয়ার্কিং ডিরেক্টরিতে প্রয়োগ করুন এবং স্ট্যাক থেকে সরান।

- `git stash pop`

→ স্ট্যাশ স্ট্যাকের শীর্ষে থাকা পরিবর্তনগুলো সম্পূর্ণভাবে বাতিল করুন।

- `git stash drop`



© 2025 Ostad. সর্বস্বত্ত্ব সংরক্ষিত।

এই ডকুমেন্টে থাকা তথ্য শুধুমাত্র শিক্ষামূলক উদ্দেশ্যে ব্যবহারের জন্য সংগ্রহ করা হয়েছে।

GitHub Education থেকে প্রাপ্ত উৎসের উপর ভিত্তি করে এই সামগ্রী প্রস্তুত করা হয়েছে এবং এর যথার্থতা নিশ্চিত করেছে Ostad। কোনো অংশ পুনঃব্যবহার বা বাণিজ্যিক উদ্দেশ্যে ব্যবহারের পূর্বে অনুমতি গ্রহণ আবশ্যিক।